



Consumiendo APIs con Python

Las APIs son una herramienta cada vez más utilizada en el mundo computacional. Gracias a estas, en la actualidad muchos sistemas **facilitan la comunicación** entre sí, en donde el interesado en algún tipo de información puede solicitarla de la manera apropiada.

Cada sistema define la forma en la que puede proveer información, incluso existen sitios que contienen documentación de cómo interactuar con su API, por ejemplo, para consultar las publicaciones en redes sociales, o para revisar el clima, el tipo de cambio de alguna divisa, el estado de una orden de compra en línea, cuando queremos ver los productos que vende una tienda, entre otros escenarios.

APIs

Como ya mencionamos, es una **forma de comunicación entre dos sistemas**, en donde una entidad solicita información en una forma específica que la otra entidad puede entender, o en la forma en la que la entidad la ofrece.

De manera general, esto funciona cuando se hace una solicitud de información, en este caso un *request*, y la API regresa una respuesta (*response*) con la información solicitada. Por lo anterior, se utilizan los términos de “consumir” una API, o llamar a la API.

Históricamente, las APIs han participado en este flujo de información desde finales de los 90s a inicios de los 2000. En su momento había dos modelos con enfoques diferentes para facilitar información: SOAP, que significa *Simple Object Access Protocol* y REST, que significa *Representational State Transfer*, cada uno con sus peculiaridades. Puedes profundizar sobre esto consultando la sección de Material adicional, al terminar esta lectura.

Interacción con Python

Empezaremos creando un primer *script* en Python para consumir una API importando la librería *requests*. Si aún no la hemos instalado, este sería el primer paso:

```
!pip install requests
import requests
```

Como lo mencionamos anteriormente, hay sitios con APIs públicas y otros que requieren algún tipo de autorización. Vamos a ver un ejemplo con una del primer tipo. Para este caso, nos comunicaremos con **randomuser.me**, un sitio que ofrece información de usuarios generados de manera aleatoria. Algunos de los datos que puede ofrecernos son: género (hombres o mujeres), nombre(s), apellido(s), dirección, correo electrónico, e incluso fotografía, entre varios otros, pero que son útiles para entender cómo funcionan las APIs.

Al consumir una API, debemos saber cómo hacerlo. Por ello, diversos sitios ofrecen documentación sobre cómo utilizarla. En el caso de **randomuser**, esto no es diferente, si deseas profundizar en ello, al final de esta lectura encontrarás los sitios recomendados para ahondar en ello.

La forma más simple de llamar a esta API es solicitando un usuario, lo que significa hacer una llamada *default* por medio de la siguiente instrucción:

```
response = requests.get("https://randomuser.me/api/")
```

Esta instrucción nos deja en la variable `response` la respuesta a esta llamada, que se encuentra en formato `.json`.

La sintaxis de JSON es muy simple y natural: el objeto se engloba entre llaves {}, y contiene parejas de la forma <clave>:<valor> separados por comas. Los valores pueden ser de varios tipos, incluyendo arreglos y otros objetos.

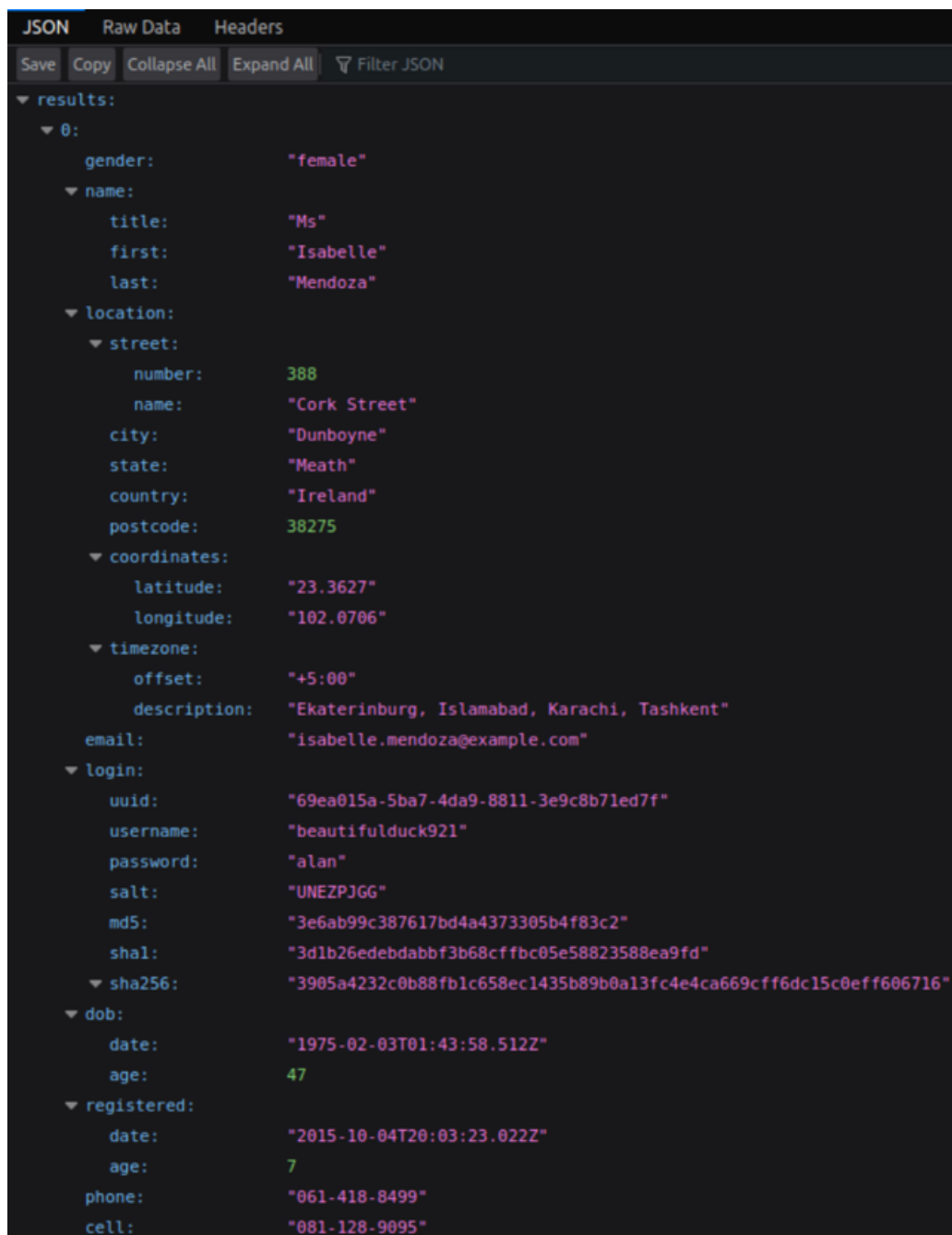
Aquí tenemos un ejemplo:

```
{
  "clave":      "C1001",
  "unidades":   6,
  "materia":    "Programación",
  "profesor":   "Juan Hernández",
  "estudiantes":[
    {
      "nombre": "Arturo",
      "apellido": "Sánchez"
    },
    {
      "nombre": "Brenda",
      "apellido": "López"
    }
  ]
}
```

Regresando al ejemplo que estamos construyendo con Python, si queremos ver el contenido de nuestra variable `response`, podemos imprimirla usando `response.text`, lo que dará como resultado un texto sin formato ni indentación. Recuerda que, al ser una API que nos regresa un usuario aleatorio, es muy probable que obtengas un resultado distinto si ejecutas el mismo código:

```
{
  "results": [
    {
      "gender": "female",
      "name": {
        "title": "Ms",
        "first": "Isabelle",
        "last": "Mendoza"
      },
      "location": {
        "street": {
          "number": 388,
          "name": "Cork Street"
        },
        "city": "Dunboyne",
        "state": "Meath",
        "country": "Ireland",
        "postcode": 38275,
        "coordinates": {
          "latitude": "23.3627",
          "longitude": "102.0706"
        },
        "timezone": {
          "offset": "+5:00",
          "description": "Ekaterinburg, Islamabad, Karachi, Tashkent"
        }
      },
      "email": "isabelle.mendoza@example.com",
      "login": {
        "uuid": "69ea015a-5ba7-4da9-8811-3e9c8b71ed7f",
        "username": "beautifulduck921",
        "password": "alan",
        "salt": "UNEZPJGG",
        "md5": "3e6ab99c387617bd4a-4373305b4f83c2",
        "sha1": "3d1b26edebdabbbf3b68cffb-c05e58823588ea9fd",
        "sha256": "3905a4232c0b88fb1c658e-c1435b89b0a13fc4e4ca669cff6dc15c0eff606716"
      },
      "dob": {
        "date": "1975-02-03T01:43:58.512Z",
        "age": 47
      },
      "registered": {
        "date": "2015-10-04T20:03:23.022Z",
        "age": 7
      },
      "phone": "061-418-8499",
      "cell": "081-128-9095",
      "id": {
        "name": "-PPS",
        "value": "8244686T"
      },
      "picture": {
        "large": "https://randomuser.me/api/portraits/women/53.jpg",
        "medium": "https://randomuser.me/api/portraits/med/women/53.jpg",
        "thumbnail": "https://randomuser.me/api/portraits/thumb/women/53.jpg"
      },
      "nat": "IE"
    }
  ],
  "info": {
    "seed": "d8deale4d2e2de8f",
    "results": 1,
    "page": 1,
    "version": "1.3"
  }
}
```

También podemos solicitar lo anterior desde cualquier navegador, obteniendo un resultado muy similar. Para ello, elige el navegador de tu preferencia, ya sea Mozilla Firefox, Google Chrome, Safari o Edge, y pega en la barra de dirección la siguiente liga: <https://randomuser.me/api/>. Verás que te regresa un JSON con la información de un usuario. Los navegadores actuales tienen **la capacidad de detectar que la respuesta se encuentre en este tipo de formato** y pueden desplegarlo de manera amigable. Observa la siguiente imagen:



Ahora veamos cómo podemos **extraer información específica de una respuesta en formato JSON**. Python tiene un buen soporte para trabajar con datos en el formato JSON en el paquete `.json`, que es parte de la librería estándar de Python, así que no es necesario instalar nada adicional para poder usarlo. De este podemos extraer información de alguno de los atributos de un objeto, por ejemplo, en el caso del usuario *random*, analizando la estructura del objeto devuelto, podemos ver que el elemento principal en la respuesta es un `result`, así que podemos extraer este elemento con la instrucción `response.json()['results']`.

Entonces, si solicitamos un usuario y luego vemos cuál es el valor del componente `results`:

```
response = requests.get("https://randomuser.me/api/")
response.json()['results']
```

Obtenemos algo como esto:

```
[{'gender': 'male',
  'name': {'title': 'Mr', 'first': 'Natão', 'last': 'Nunes'},
  'location': {'street': {'number': 3476, 'name': 'Rua Dezenove de Outubro'}},
  'city': 'Rio de Janeiro',
  'state': 'Amapá',
  'country': 'Brazil',
  'postcode': 78666,
```

```
'coordinates': {'latitude': '-76.2497', 'longitude':  
'-93.0421'},  
  
  'timezone': {'offset': '+7:00', 'description': 'Bangkok,  
Hanoi, Jakarta'}}},  
  
  'email': 'natao.nunes@example.com',  
  
  'login': {'uuid': 'bbd53b1c-d8ed-47ba-b9ca-03a59b04a32e',  
  
    'username': 'smallmouse109',  
  
    'password': 'morgan',  
  
    'salt': '3F62CTPE',  
  
    'md5': 'b5020eebbfaa9e3b4145601044abc2e9',  
  
    'sha1': 'a960f7feb1b37a4d9ee54de2019cc425bedd1a76',  
  
    'sha256': 'd50c55b163ec4c683ec3d490149b97b3cfd-  
b4a1462e48b93de1224a2afe7e787'},  
  
  'dob': {'date': '1956-05-26T23:24:25.375Z', 'age': 68},  
  
  'registered': {'date': '2015-06-25T16:51:52.712Z', 'age':  
9},  
  
  'phone': '(57) 0262-1804',  
  
  'cell': '(56) 7984-1871',
```

```
'id': {'name': 'CPF', 'value': '286.907.231-92'},  
  
  'picture': {'large': 'https://randomuser.me/api/portraits/  
men/31.jpg',  
  
    'medium': 'https://randomuser.me/api/portraits/med/  
men/31.jpg',  
  
    'thumbnail': 'https://randomuser.me/api/portraits/thumb/  
men/31.jpg'},  
  
  'nat': 'BR']}]
```

Un arreglo que también podemos ver como una lista de un solo elemento, así que podemos extraer el primer elemento de la siguiente manera:

```
response.json()['results'][0]
```

También podemos extraer otros datos, como el correo electrónico de ese primer resultado:

```
response.json()['results'][0]['email']
```

O podríamos consultar el nombre de este mismo escenario:

```
response.json()['results'][0]['name']['first']
```

En este caso, el nombre es otro valor compuesto, la respuesta sería algo como esto: `{ 'title': 'Mr', 'first': 'Natão', 'last': 'Nunes' }` y podemos extraer cada una de sus partes:

```
response.json()[ 'results' ][0][ 'name' ][ 'title' ]
response.json()[ 'results' ][0][ 'name' ][ 'last' ]
response.json()[ 'results' ][0][ 'name' ][ 'first' ]
```

La documentación de la API *random user* declara que podemos **pedir más de un usuario; esto se hace pasando un parámetro en la URL indicando cuántos usuarios queremos**:

```
response = requests.get("https://randomuser.me/api/?results=20")
```

Es importante destacar que, al ser un sitio que nos proporciona el uso gratuito de una API, no debemos exceder ciertos límites: si bien nos permite generar una cantidad de usuarios, debemos mantener las solicitudes moderadas para no ser restringidos.

Para poder **crear listas de usuarios** o algo parecido, veamos un ejemplo en donde hacemos una petición a la API, declaramos una lista e incluimos en esta los 20 nombres recuperados:

```
response = requests.get("https://randomuser.me/api/?results=20")
```

```
list = []
```

```
for i in range (20):
```

```
list.append(response.json() ['results'] [i] ['name'])

print(list)
```

El resultado de ejecutar este código da una lista de 20 objetos, donde cada uno contiene título, nombre y apellido:

```
[{'title': 'Miss', 'first': 'Constanze', 'last': 'Sager'},
{'title': 'Mr', 'first': 'Connor', 'last': 'Davidson'},
{'title': 'Miss', 'first': 'Vanesa', 'last': 'Oliveira'},
{'title': 'Miss', 'first': 'Kelin', 'last': 'Araújo'}, {'tit-
le': 'Mr', 'first': 'Chaïm', 'last': 'Hulzebosch'}, {'tit-
le': 'Mr', 'first': 'Vinay', 'last': 'Heeren'}, {'title':
'Ms', 'first': 'Colleen', 'last': 'Holt'}, {'title': 'Mr',
'first': 'Eduardo', 'last': 'Mitchell'}, {'title': 'Mr',
'first': 'Veeti', 'last': 'Eskola'}, {'title': 'Mr', 'first':
'Liam', 'last': 'Barnaby'}, {'title': 'Mrs', 'first': 'Josefi-
na', 'last': 'Castillo'}, {'title': 'Miss', 'first': 'Elis',
'last': 'Nunes'}, {'title': 'Mrs', 'first': 'Brielle',
'last': 'Scott'}, {'title': 'Mr', 'first': 'ایدرپ', 'last':
'یرفعج'}, {'title': 'Miss', 'first': 'Angelina', 'last': 'Nu-
nes'}, {'title': 'Mr', 'first': 'George', 'last': 'Jackson'},
{'title': 'Mrs', 'first': 'Lucia', 'last': 'Sæverud'}, {'tit-
le': 'Mr', 'first': 'Arnold', 'last': 'Morrison'}, {'tit-
le': 'Miss', 'first': 'Ashley', 'last': 'Morales'}, {'title':
'Mrs', 'first': 'Sofia', 'last': 'Brady'}]
```

Si quisiéramos recuperar únicamente los nombres, podríamos declarar algo como:

```
names = []

for i in range (20):

    names.append(response.json() ['results'] [i] ['name'] ['first'])

print(names)
```

Obteniendo el siguiente resultado:

```
['Constanze', 'Connor', 'Vanesa', 'Kelin', 'Chaim', 'Vinay',
'Colleen', 'Eduardo', 'Veeti', 'Liam', 'Josefina', 'Elis',
'Brielle', 'ایدرې', 'Angelina', 'George', 'Lucia', 'Arnold',
'Ashley', 'Sofia']
```

También podríamos crear una lista de nombres con apellidos:

```
names = []

for i in range (20):

    names.append(response.json() ['results'] [i] ['name'] ['first'] +
" " +response.json() ['results'] [i] ['name'] ['last'])

print(names)
```

Obteniendo el siguiente resultado:

```
['Constanze Sager', 'Connor Davidson', 'Vanesa Oliveira',  
'Kelín Araújo', 'Chaïm Hulzebosch', 'Vinay Heeren', 'Colleen  
Holt', 'Eduardo Mitchell', 'Veeti Eskola', 'Liam Barnaby',  
'Josefina Castillo', 'Elis Nunes', 'Brielle Scott', 'ایدرېب',  
'یرفعج', 'Angelina Nunes', 'George Jackson', 'Lucia Sæverud',  
'Arnold Morrison', 'Ashley Morales', 'Sofia Brady']
```

Recuerda que al interactuar con una API pública solo estamos consumiendo datos de la forma en que el servicio nos lo permite, pero también podemos conectarnos a algunas APIs en las que se requiera algún tipo de autenticación del usuario, y se pueden llevar a cabo más acciones, por ejemplo, el eliminado de datos, actualizaciones, o incluso inserciones.

Sin embargo, esto no termina aún. Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.



Glosario

JavaScript: Lenguaje de programación orientado a programación de la web.

JSON: Son las siglas de *Javascript Object Notation* (Notación de Objetos de JavaScript).

Material adicional

Curry, C. (19 de noviembre de 2020). *REST API Crash Course - Introduction + Full Python API Tutorial* [Curso intensivo sobre API REST: Introducción + Tutorial completo sobre API de Python] [Video]. YouTube. Recuperado el 30 de marzo de 2025, de <https://www.youtube.com/watch?v=qbLc5a9jdXo>

Random User Generator. (s.f.). *Random User API documentation* [Documentación de la API Random User]. Recuperado el 30 de marzo de 2025, de <https://randomuser.me/documentation>

RandomUser. (s. f.). *Random User Generator Documentation* [Documentación del generador de usuarios aleatorios]. Recuperado el 30 de marzo de 2025, de <https://randomuser.me/documentation>

RandomUser. (s. f.). *Random User API* [API de usuario aleatorio]. Recuperado el 30 de marzo de 2025, de <https://randomuser.me/api/>

The Cat API. (s. f.). *Breeds Endpoint Documentation* [Documentación de puntos finales de Breeds]. Recuperado el 30 de marzo de 2025, de <https://api.thecatapi.com/v1/breeds/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo de 2025 y su disponibilidad depende del autor.

Bibliografía

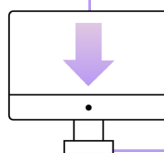
Richardson, L. (s.f). *Beautiful Soup Documentation*. Crummy. Recuperado el 30 de marzo de 2025, de <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

W3C. (s. f.). *World Wide Web Consortium (W3C)* [Consortio de la World Wide Web]. Recuperado el 30 de marzo de 2025, de <https://www.w3.org/>

Wikipedia. (s. f.). *Anexo: Películas de Walt Disney Pictures*. Recuperado el 30 de marzo de 2025, de https://es.wikipedia.org/wiki/Anexo:Pel%C3%ADculas_de_Walt_Disney_Pictures

Wikipedia. (s. f.). *Titans (2018 TV series)* [Titans (serie de televisión 2018)]. Recuperado el 30 de marzo de 2025, de [https://en.wikipedia.org/wiki/Titans_\(2018_TV_series\)](https://en.wikipedia.org/wiki/Titans_(2018_TV_series))

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 30 de marzo de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.